



唐山学院

毕业设计（论文）

基于 SpringBoot 的程序设计评测系统的 设计与实现

姓 名：李融荣

学 号：2240010316

院 系：人工智能学院

专 业：计算机科学与技术

指导教师：王雅坤

二〇二五年五月一日

毕业设计（论文）原创性声明

本人所提交的毕业设计（论文）基于 Spring Boot 的程序设计评测系统的设计与实现，是在王雅坤导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

论文作者（签名）：

指导教师（签名）：

2025 年 5 月 21 日

2025 年 5 月 21 日

毕业设计（论文）版权使用授权书

本毕业设计（论文）作者完全了解唐山学院有权保留并向国家有关部门或机构送交毕业设计（论文）的复印件和磁盘，允许毕业设计（论文）被查阅和借阅。本人授权唐山学院可以将毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编毕业设计（论文）。

保密的毕业设计（论文）在_____年解密后适用本授权书。

论文作者（签名）：

指导教师（签名）：

年 月 日

年 月 日

摘 要

针对传统程序设计课程中作业批改效率低、反馈不及时、评价标准不统一以及教学过程数据难以沉淀等问题，设计并实现了一套基于 Spring Boot 的程序设计评测系统。系统采用前后端分离的 B/S 架构，后端以 Spring Boot 为核心实现业务服务与权限控制，前端基于 Vue 构建交互界面，数据库采用 MySQL 完成数据持久化，并结合自动判题机制实现代码提交、编译运行、结果比对与评测反馈。系统面向学生、教师和管理员三类用户，支持题库训练、在线提交、自动评测、竞赛组织、成绩统计、讨论交流以及系统等功能，形成了较为完整的教学闭环。通过系统设计与实现可知，该平台能够提高程序设计课程的教学效率，增强评测过程的客观性与及时性，同时为教师掌握学生学习情况、开展教学分析提供数据支持。测试结果表明，系统主要功能运行稳定，能够满足课程实践教学与基础竞赛场景的应用需求，具有一定的推广价值和实际意义。

关键词：程序设计评测系统；自动判题；SpringBoot；MySQL 数据库；Vue

ABSTRACT

To address the issues in traditional programming courses—such as low efficiency in assignment grading, delayed feedback, inconsistent evaluation standards, and difficulty in accumulating teaching data—a programming evaluation system based on Spring Boot has been designed and implemented. The system adopts a front-end and back-end separated B/S architecture. The back-end is built on Spring Boot to implement business logic and access control, while the front-end is developed using Vue to construct interactive user interfaces. MySQL is used for data persistence. In addition, an automatic judging mechanism is integrated to support code submission, compilation and execution, result comparison, and evaluation feedback.

The system is designed for three types of users: students, teachers, and administrators. It supports functionalities such as problem set training, online submission, automatic evaluation, contest organization, performance statistics, discussion and communication, and system management, thereby forming a relatively complete teaching closed-loop. Through the design and implementation of this system, it is demonstrated that the platform can improve the teaching efficiency of programming courses, enhance the objectivity and timeliness of the evaluation process, and provide data support for teachers to monitor students' learning progress and conduct teaching analysis. Experimental results show that the system operates stably in its core functions, meets the requirements of course practice teaching and basic competition scenarios, and possesses certain practical significance and potential for wider application.

Key words: Programming Evaluation System; Automatic Judging; Spring Boot; MySQL Database; Vue
spring boot; vue; contest management

目 录

第 1 章 引言.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	1
1.3 研究内容与目标.....	2
1.4 论文结构安排.....	2
第 2 章 开发技术介绍.....	3
2.1 系统架构与开发模式.....	3
2.1.1 BS 架构特点.....	3
2.1.2 前后端分离模式.....	3
2.2 Spring Boot 技术.....	3
2.3 Vue 技术.....	4
2.4 MySQL 与 Docker 技术.....	5
第 3 章 系统分析.....	6
3.1 功能需求分析.....	6
3.1.1 学生端需求.....	6
3.1.2 教师端需求.....	6
3.1.3 管理员端需求.....	6
3.2 非功能需求分析.....	7
3.3 可行性分析.....	7
第 4 章 系统设计.....	8
4.1 系统总体设计.....	8
4.2 功能模块设计.....	8
4.3 数据库设计.....	9
4.3.1 数据库概念结构设计.....	9
4.3.2 数据库逻辑结构设计.....	9
4.3.3 数据库表结构设计.....	9
4.4 关键流程设计.....	10
第 5 章 系统实现.....	11
5.1 用户认证与权限模块实现.....	11
5.2 题库与判题模块实现.....	11
5.3 竞赛管理模块实现.....	12
5.4 讨论区模块实现.....	12
5.5 后台管理与统计实现.....	12
第 6 章 系统测试.....	14

6.1 测试环境与测试方法.....	14
6.2 功能测试与结果分析.....	14
6.3 性能测试与安全测试.....	14
6.4 测试结论.....	15
第 7 章 结论.....	16
参考文献.....	17
致 谢.....	18
附 录.....	19

第 1 章 引言

1.1 研究背景与意义

程序设计课程是计算机类专业核心课程。传统人工批改模式在大规模教学场景下存在反馈滞后、评价主观、过程数据缺失等问题。在线评测系统通过标准化测试点和自动判题机制，可显著提升教学效率与评价客观性，并帮助学生形成“提交—反馈—修正”的高频训练闭环^[1]。

随着高校信息化教学水平不断提升，程序设计课程已经不再局限于课堂讲授和课后纸质作业的传统模式。学生需要更多可重复练习的平台，教师也需要更高效的过程管理手段，以便及时掌握学生对语法、算法和编程规范的掌握情况。在这一背景下，能够支持在线提交、自动评测、结果反馈和成绩统计的在线评测系统，逐渐成为程序设计教学中的重要基础设施。

从教学目标角度看，程序设计课程不仅要求学生掌握语言语法，更强调问题分析、算法设计、编码实现和调试优化等综合能力。在线评测系统可以将题库训练、过程反馈和竞赛实践整合到同一平台中，使学生在反复提交和修正中不断提升分析与实现能力。同时，系统沉淀下来的提交记录、错误类型和成绩变化趋势，也为教师改进教学方案提供了数据依据。因此，研究并实现一套面向教学场景的程序设计评测系统，具有明显的教学价值与现实意义。

1.2 国内外研究现状

国内外在线评测平台已广泛应用于课程教学与程序设计竞赛。研究重点主要集中在判题准确性、多语言支持、并发处理能力、反作弊与可视化统计等方向^{[2][3]}。面向高校教学的系统通常更关注可用性、低成本部署与教学流程贴合度。

国外在线评测平台起步较早，较多应用于 ACM、ICPC 等程序设计竞赛以及开放式编程训练场景。相关平台通常具备题目管理、判题服务、排行榜和用户社区等功能，强调高并发支持与跨语言兼容能力。国内高校近年来也不断将在线评测系统引入课程教学，形成了从作业布置、平时训练到课程竞赛的多层次应用模式。

现有研究在评测机制、系统架构与教学应用方面均取得了一定成果，但仍存在一些不足。一方面，部分系统更偏向竞赛场景，对教学过程中的成绩分析、角色协作与日常管理

支持不够充分；另一方面，一些教学系统在自动判题、安全隔离和扩展性方面仍有提升空间。基于此，本文结合高校程序设计课程的实际需求，设计并实现一套兼顾教学训练、竞赛组织和系统管理的在线评测平台，以期在功能完整性和教学适配性之间取得较好的平衡。

1.3 研究内容与目标

围绕“程序设计课程教学”目标，完成如下研究与实现：

1. 设计三角业务模型（学生、教师、管理员）。
2. 实现题库管理、在线提交、自动评测、竞赛管理、讨论交流等核心模块。
3. 构建稳定可扩展的前后端分离架构与数据库模型。
4. 通过测试验证系统可用性，形成可用于答辩展示的工程成果。

本文的研究内容不仅包括系统功能实现，还包括对系统整体架构、业务流程、数据模型和测试验证方法的系统性整理。通过需求分析明确平台的角色定位和业务边界，在系统设计阶段完成模块划分、数据库建模与关键流程设计，在系统实现阶段对用户认证、题库管理、自动评测、竞赛管理和讨论交流等模块进行落地开发，最终通过功能测试与场景验证评估系统是否满足预期目标。

在目标层面，本文希望构建一个能够服务课程训练与基础竞赛场景的程序设计评测系统。一方面，系统应满足学生日常练习、教师组织教学和管理员运行维护的基本需求；另一方面，系统还应具有较好的可扩展性，为后续增加题目导入、更多语言支持、并发评测优化和学习数据分析等功能保留空间。

1.4 论文结构安排

全文结构包括：开发技术介绍、系统分析、系统设计、系统实现、系统测试、结论与展望。

其中，第 1 章主要介绍课题研究背景、国内外研究现状、研究内容和论文结构；第 2 章介绍系统采用的主要开发技术与实现环境；第 3 章从角色和业务角度分析系统需求；第 4 章对系统总体架构、功能模块、数据库和关键流程进行设计；第 5 章阐述系统各核心模块的实现过程；第 6 章对系统进行测试并分析结果；第 7 章总结全文工作并提出后续改进方向。

第 2 章 开发技术介绍

2.1 系统架构与开发模式

系统采用前后端分离 B/S 架构。前端负责页面呈现与交互，后端负责业务逻辑、权限控制与数据访问。

在整体实现上，系统通过浏览器访问前端页面，用户提交的操作请求由前端封装为 HTTP 请求发送至后端接口，后端再完成权限校验、业务处理、数据读写和结果返回。对于代码评测这类计算型任务，系统将提交信息写入业务链路后，再由判题服务执行编译、运行和结果比对，从而将普通业务处理与判题任务执行进行适度解耦。

该架构方式适合教学类系统的逐步迭代开发。一方面，前端界面可以围绕题库、竞赛、讨论区和后台管理等页面独立演进；另一方面，后端也可以基于控制层、服务层和数据访问层逐步扩展接口和业务逻辑，降低系统后续维护与功能升级的难度。

2.1.1 BS 架构特点

B/S 架构具备部署简单、更新集中、跨平台访问等优势，适合教学管理类系统。

与传统 C/S 架构相比，B/S 架构不需要在每台终端安装独立客户端，用户只需通过浏览器即可访问系统功能。这一特点对于高校教学环境尤为重要，因为实验室电脑、学生个人电脑和教师办公设备往往存在差异，采用浏览器访问方式可以有效降低环境适配成本。

同时，B/S 架构在系统更新时更具优势。功能调整或缺陷修复主要集中在服务器端和前端部署端完成，用户无需重复安装或更新客户端程序，能够提升系统运维效率。对于需要在学期中持续迭代的教学平台而言，这种集中式维护模式具有较高的实用价值。

2.1.2 前后端分离模式

前端通过 RESTful API 与后端通信。该模式有利于模块解耦、并行开发和后续维护。

前后端分离模式使界面层与业务层职责更加清晰。前端主要关注页面布局、状态管理、交互逻辑与用户体验，后端则专注于权限认证、业务规则、数据访问和结果返回。通过约定统一的接口规范，双方可以并行开发，从而提高开发效率。

在本系统中，这种模式还提升了后续扩展能力。例如，若未来需要增加移动端、小程序端或替换部分前端页面实现，只要后端接口保持兼容，整体业务逻辑无需大幅改动；同样，后端新增统计分析接口时，也不会直接影响既有页面结构，系统耦合度相对更低。

2.2 Spring Boot 技术

Spring Boot 提供自动配置与组件化支持，能够快速搭建企业级 Web 服务。系统基于 Controller、Service、Mapper 分层组织代码，提升可维护性。

Spring Boot 是当前 Java Web 开发中应用广泛的主流框架，具备自动配置、依赖整合和快速启动等特点。通过引入 Spring Boot，开发者可以在较少配置的前提下完成 Web 服务搭建，并配合 Spring Security、Validation、数据访问框架等生态组件构建完整的业务系统。

本系统后端以 Spring Boot 2.7.18 为基础，并结合 Spring Security、MyBatis-Plus、Flyway、Redis 和 JWT 等技术实现认证授权、数据库访问、结构迁移、缓存支持与登录态管理。系统采用 Controller、Service、Mapper 分层方式组织代码，其中 Controller 层负责请求接收与参数转发，Service 层负责业务逻辑处理，Mapper 层负责与数据库交互，这种分层结构有利于提高系统可读性与可维护性。

在教学系统场景中，Spring Boot 的优势主要体现在开发效率与扩展能力上。无论是新增题库接口、调整竞赛逻辑，还是补充日志与监控能力，都可以在既有分层结构上逐步扩展，不必推翻整体架构。对于毕业设计项目而言，这种技术路线既成熟稳定，也便于后续继续迭代完善。

结合项目代码结构，后端以 Spring Boot 2.7.18 为核心框架，并集成 Spring Security、Validation、MyBatis-Plus、Flyway、Redis 与 JWT 等组件，形成“接口接入、业务处理、数据访问、权限控制、运行监测”较为完整的企业级开发基础设施。

2.3 Vue 技术

Vue3 提供响应式数据驱动和组件化能力，配合 Router、Pinia 与 Axios 形成完整前端工程体系。

Vue 是轻量级前端框架，具备响应式更新和组件化开发等特点。Vue3 在性能、逻辑复用和工程化方面进行了进一步增强，适合构建交互较多的单页应用。在本系统中，前端页面包括登录注册、题库列表、题目详情、提交记录、竞赛页面、讨论区以及后台管理页面，采用 Vue 进行实现能够较好地支撑页面状态切换和数据联动。

为了形成完整的前端开发体系，系统还结合 Vue Router 处理页面路由跳转，使用 Pinia 进行状态管理，并通过 Axios 统一处理接口请求与响应拦截。这样既便于维护登录状态和页面数据，也便于对请求失败、权限失效等场景进行统一处理。

从用户体验角度看，Vue 的组件化机制使题目卡片、排行榜表格、提交状态标签、评论列表等界面元素可复用性更高，能够在保证一致性的同时提高开发效率。对于教学平台这类页面较多、交互相对复杂的系统而言，Vue 技术具有较好的适配性。

前端工程基于 Vue 3 生态实现，配合 Vue Router、Pinia、Axios 与 Element Plus 组

织页面、状态与接口调用流程，能够较好支撑题库检索、提交列表、竞赛页面和讨论交流等交互场景。

2.4 MySQL 与 Docker 技术

MySQL 用于业务数据持久化；Docker 用于判题容器隔离执行，限制代码运行资源并降低安全风险。

MySQL 是成熟的关系型数据库管理系统，具备良好的稳定性和较强的数据管理能力。系统中用户信息、题目信息、测试数据、提交记录、竞赛数据、讨论区内容和系统配置等结构化数据均存储在 MySQL 中。通过合理设计主键、外键与索引，可以保证数据一致性，并提升题目查询、成绩统计和提交记录检索等业务场景下的访问效率。

判题模块涉及用户代码的编译与运行，如果直接在业务服务器环境中执行，可能带来资源占用、环境污染和安全风险。因此，系统引入 Docker 容器技术，为提交代码提供相对隔离的运行环境。容器内可限制时间、内存和运行命令范围，在一定程度上提升判题过程的安全性可控性。

将 MySQL 与 Docker 同时引入系统后，前者负责业务数据可靠存储，后者负责评测运行环境隔离，二者在系统中承担着不同但都十分关键的角色，共同保障了平台功能的稳定实现。

第 3 章 系统分析

3.1 功能需求分析

3.1.1 学生端需求

学生需支持注册登录、题目练习、在线提交、结果查看、竞赛报名、帖子互动等能力。

学生作为系统的主要使用者，需要在平台中完成从练习到反馈的完整学习过程。因此，学生端首先应具备注册、登录、个人信息查看与修改等基础能力，以满足身份识别和个性化使用需求。在题库模块中，学生应能够浏览题目列表，查看题目描述、输入输出要求、样例数据和难度标签，并根据自身学习进度选择适合题目进行练习。

在提交评测方面，学生需要能够在线编写或粘贴代码，选择编程语言并发起提交。系统应返回编译错误、运行错误、答案错误、时间超限、内存超限和通过等不同评测结果，并允许学生查看历史提交记录，以便进行针对性修改和反复练习。此外，学生还需要通过竞赛模块参与课程竞赛、查看排行榜和历史成绩，并借助讨论区进行提问、交流与经验分享。

3.1.2 教师端需求

教师需支持题目维护、测试数据配置、竞赛创建与成绩导出，以服务课程组织。

教师是教学活动的组织者和评价者，因此教师端需求更强调教学管理能力。教师需要能够发布和维护题目，设置题目描述、样例数据、测试点、时间限制与内存限制，以确保题目既符合教学目标，又能支撑自动判题逻辑。同时，教师还应具备对提交情况进行查看与分析的能力，以掌握学生对不同知识点的学习情况。

在课程组织场景下，教师还需要创建并管理竞赛，配置竞赛时间、题目集合、排行榜规则和罚时策略等信息，用于课堂训练、阶段测验或基础竞赛。为了便于教学评估，系统还应支持成绩统计和数据导出，使教师能够将成绩信息整理后用于课堂总结和过程考核。

3.1.3 管理员端需求

管理员需支持用户与角色管理、系统配置、日志审计、运行状态监控等能力。

管理员负责系统的整体运行维护，其需求重点在于系统级配置和安全管理。管理员需要对学生、教师和管理员账户进行维护，处理角色分配、用户状态管理以及异常账号排查等问题，以保证平台基础数据的准确性。

此外，管理员还需要查看系统配置、运行状态和操作日志，以便发现潜在故障并及时

处理。例如，当判题服务异常、题目数据不完整或系统接口出现访问错误时，管理员应能够借助监控信息和日志记录快速定位问题。由此可见，管理员端在系统中承担着保障稳定运行和维护平台秩序的重要职责。

3.2 非功能需求分析

系统需满足安全性、稳定性、可扩展性与易用性要求。包括接口鉴权、异常兜底、评测隔离、中文化交互等。

安全性是在线评测系统的重要非功能需求之一。由于系统涉及用户登录、代码提交和后台管理等功能，因此必须对接口访问进行身份校验和角色控制，避免未授权用户访问敏感资源。同时，判题过程需要对用户提交代码进行隔离执行，防止恶意代码直接影响服务器环境。

稳定性方面，系统应能够保证核心业务流程连续可用，例如登录认证、题目浏览、提交评测和竞赛统计等功能不能频繁中断。对于异常输入、服务超时和判题失败等情况，系统需要具备一定的错误处理与兜底机制，以提升整体可用性。

可扩展性和易用性同样不可忽视。系统应便于后续扩展更多语言支持、题目导入方式和数据分析能力，同时界面设计需要尽量清晰直观，使学生和教师能够快速理解和使用系统功能。对于教学平台而言，只有在“功能可用”和“操作易用”之间取得平衡，系统才能真正服务于日常教学。

3.3 可行性分析

从技术、经济、实施三方面评估，项目具备可行性：技术栈成熟，成本可控，阶段目标清晰，已完成核心功能实现。

从技术可行性来看，系统所采用的 Spring Boot、Vue、MySQL、Redis 和 Docker 等技术均较为成熟，拥有较完善的文档资料与社区支持，能够满足程序设计评测系统的开发需求。尤其是 Spring Boot 与 Vue 的前后端分离方案，已经在大量管理系统和教学平台中得到验证，具有较高的工程可行性。

从经济可行性来看，本系统开发所需的软件环境大多为开源或常见教学开发工具，服务器资源需求也相对可控，适合在高校课程项目和毕业设计场景中实施。系统不依赖昂贵的商业平台，部署和维护成本较低。

从实施可行性来看，项目开发目标明确，模块边界较清晰，且当前已经完成用户、题库、判题、竞赛、讨论和后台管理等核心功能，实现了一个可运行、可展示、可继续迭代的原型系统。因此，无论从开发条件还是阶段成果看，本课题均具备较好的实施基础。

第 4 章 系统设计

4.1 系统总体设计

系统分为表示层、业务层、数据访问层和数据层。判题服务独立处理评测任务，业务服务统一维护提交状态与结果。

在总体结构上，系统以前后端分离模式组织实现。前端作为表示层，负责页面展示、数据交互和用户操作引导；后端作为业务核心，负责接口接收、权限控制、业务流程处理以及数据持久化。这样的结构能够使不同模块职责明确，便于系统后续分工开发与功能扩展。

后端内部进一步采用分层设计思想，将控制层、服务层和数据访问层进行划分。控制层负责接收请求、校验参数并返回统一格式的数据；服务层负责处理具体业务逻辑，如注册登录、题目管理、竞赛统计和讨论回复；数据访问层负责执行数据库操作。对于代码评测任务，系统通过独立的判题链路执行编译、运行和结果比对，再将评测结果写回业务数据表，从而保证判题过程与普通业务流程相对独立。

这种总体设计兼顾了系统结构清晰性和功能完整性，为后续数据库设计、模块设计和实现细节展开提供了基础。

4.2 功能模块设计

1. 用户认证与权限模块。
2. 题库与测试数据模块。
3. 在线判题模块。
4. 竞赛与排行榜模块。
5. 讨论区模块。
6. 系统配置与日志模块。

用户认证与权限模块主要负责注册、登录、身份识别和角色控制，是系统安全运行的基础。题库与测试数据模块负责题目维护、测试点配置和题目展示，是学生训练和教师出题的核心支撑。在线判题模块则围绕代码提交、编译运行、结果比对和状态回写等流程展开，是系统最关键的特色功能之一。

竞赛与排行榜模块用于支撑阶段训练赛、课程竞赛等教学活动，能够提升学生参与度，并为教师提供更具挑战性的实践组织方式。讨论区模块则为学生交流题解思路、提出问题和分享经验提供空间，有助于形成学习共同体。系统配置与日志模块主要服务于管理员，

用于维护平台运行参数、记录操作行为和辅助问题排查。

各功能模块之间并非孤立存在，而是通过统一用户体系和业务数据相互关联。例如，学生在题库中提交代码后会生成提交记录并参与成绩统计，竞赛成绩又会反映到排行榜中，

讨论区内容也可以围绕题目展开交流。正是这些模块间的协同，构成了完整的程序设计教学闭环。

4.3 数据库设计

4.3.1 数据库概念结构设计

核心实体包括用户、题目、测试点、提交、竞赛、帖子与评论等。

数据库概念结构设计的目标是从业务需求出发，抽象出系统运行所需的主要实体及其关系。结合本系统实际功能，可以将用户划分为学生、教师和管理员三类角色，同时围绕题目训练与评测过程设计题目、测试点、提交记录和判题结果等实体。

在竞赛业务中，还需要设计竞赛、竞赛题目关联、参赛记录和成绩统计等实体，以支撑竞赛创建、报名、比赛提交和排行榜生成等功能。讨论区业务则需要帖子和评论实体，用于记录交流内容和回复关系。此外，系统运行维护还涉及系统配置和管理员操作日志等实体，用于支撑后台管理与问题追踪。

依据当前项目 ER 设计，核心实体进一步细化为用户、学生档案、教师档案、管理员档案、题目、测试点、提交记录、判题结果、竞赛、竞赛题目关联、竞赛参与、竞赛成绩、讨论帖子、讨论评论、系统配置和管理员操作日志等对象。

4.3.2 数据库逻辑结构设计

关键关系如下：用户-提交（一对多）、题目-测试点（一对多）、竞赛-题目（多对多）、帖子-评论（一对多）。

在逻辑结构层面，用户与提交记录之间属于一对多关系，一个用户可以产生多条代码提交记录；题目与测试点之间同样是一对多关系，一个题目通常对应多组测试数据。竞赛与题目之间属于多对多关系，因此需要通过中间表保存关联信息。帖子与评论之间构成一对多关系，而评论本身还可以通过父评论字段形成回复层级。

此外，竞赛与参赛用户之间也属于多对多关系，通常通过参赛表或成绩表进行关联；判题结果与提交记录之间则存在紧密映射关系，用于记录评测状态、耗时、内存和错误信息等内容。通过这些逻辑关系设计，数据库能够较完整地表达系统中的主要业务流程。

4.3.3 数据库表结构设计

user: 主要字段包括 id, username, role, status, 说明为用户信息。

problem: 主要字段包括 id, title, difficulty, time_limit, 说明为题目信息。

test_case: 主要字段包括 `id`, `problem_id`, `input_data`, `output_data`, 说明为测试点数据。

submission: 主要字段包括 `id`, `user_id`, `problem_id`, `language`, `verdict`, 说明为提交结果。

contest: 主要字段包括 `id`, `title`, `start_time`, `end_time`, 说明为竞赛信息。

discussion_post: 主要字段包括 id, user_id, title, content, 说明为帖子信息。

表格导出到正式稿时采用三线表。

在实际实现中, 用户表主要保存账号、角色、状态等基础信息; 题目表保存题目标题、描述、限制条件和难度标签; 测试点表保存标准输入输出和示例标识; 提交表与判题结果表共同记录用户代码提交及评测反馈信息。竞赛相关表用于保存竞赛配置、题目关联、报名记录和排行榜统计结果, 讨论相关表则用于记录帖子和评论内容。

数据库表结构设计既要满足业务功能需求, 也要兼顾查询效率与数据一致性。因此, 系统在主键设计、外键关联和索引配置上进行了基础考虑, 使题目查询、提交检索、竞赛统计和讨论内容展示等高频场景能够保持较好的访问性能。

4.4 关键流程设计

提交评测流程: 提交代码 → 创建任务 → 沙箱编译运行 → 输出比对 → 写回结果。

竞赛流程: 创建竞赛 → 关联题目 → 学生报名 → 赛中提交 → 榜单计算。

在提交评测流程中, 学生首先在题目页面选择语言并提交代码, 后端接收请求后生成提交记录并保存初始状态。随后系统调用判题服务, 在隔离环境中执行编译和运行操作, 并将程序输出与标准结果进行比对, 最终生成通过、答案错误、运行错误、超时等评测结论, 再将结果回写到数据库并反馈给前端页面。该流程体现了系统“提交即反馈”的教学特征。

竞赛流程则更强调时间约束和成绩统计逻辑。教师或管理员创建竞赛后, 需要配置竞赛时间、题目列表和相关规则, 学生在竞赛期间完成报名并进行提交。系统根据通过题数、提交次数和罚时规则动态生成排行榜, 并在比赛结束后保留成绩数据, 供教师查看和分析。这一流程使系统不仅能够满足日常训练, 也能够支撑课程竞赛场景。

第 5 章 系统实现

5.1 用户认证与权限模块实现

系统基于 JWT 实现登录态维护，拦截器完成 Token 校验与角色鉴权。游客可浏览公开页面，提交和管理操作需登录。

在实现过程中，系统通过认证控制器接收用户登录请求，完成账号校验后签发 JWT 令牌。前端在用户登录成功后保存令牌信息，并在后续请求中携带该令牌访问后端接口。后端则通过拦截器或过滤链对令牌进行解析和校验，识别当前访问用户的身份信息与角色类型。

为了保证不同角色功能边界清晰，系统对学生、教师和管理员的访问权限进行区分。学生主要访问训练、竞赛和讨论相关功能；教师可维护题目、测试点和竞赛信息；管理员则负责账户管理、系统配置和日志查看。该模块的实现为后续各业务模块提供了统一的身份认证基础，也提升了系统整体安全性。

从控制器层看，系统通过 AuthController、UserController 与相关拦截逻辑完成注册、登录、用户信息查询以及基于角色的访问控制，保证学生、教师、管理员在不同功能入口下具有清晰的权限边界。

5.2 题库与判题模块实现

实现题目增删改查、测试点维护、在线提交与结果回显。判题结果支持 AC、WA、TLE、MLE、RE、CE 等状态。

题库模块主要围绕题目管理与展示展开。在后端实现中，系统提供题目列表、题目详情、题目新增、编辑和删除等接口，并支持教师和管理员维护测试点数据。前端页面则根据题目难度、标签和状态对数据进行展示，方便学生按需检索和练习。

判题模块是系统最核心的实现部分。用户提交代码后，系统会生成提交记录，并触发判题服务执行编译、运行和输出比对操作。针对不同语言，系统分别调用对应的编译器或解释器，在受限环境中完成执行过程，并记录运行耗时、内存使用和错误信息。最终评测结果会同步至提交记录中，以便学生查看详情并进行后续改进。

从实际工程角度看，题库与判题模块不仅承担核心教学功能，还直接影响用户对系统的使用体验。如果评测结果反馈不及时或状态定义不明确，学生难以快速定位问题。因此，本系统在状态划分、结果回写和提交详情展示上进行了较为明确的实现设计。

题库与判题链路由 ProblemController、SubmissionController、TestCaseController 以及 JudgeService 等模块协同完成，能够实现题目查询、测试点维护、提交轮询、结果持久化

与状态回写等关键流程。

5.3 竞赛管理模块实现

实现竞赛创建、编辑、删除、报名、排行榜统计。支持按通过题数与罚时计算排名。

竞赛管理模块主要服务于课程训练赛和基础竞赛组织场景。系统允许教师或管理员创建竞赛，配置竞赛名称、起止时间、题目集合、排行榜冻结时间和罚时规则等信息，并对竞赛进行编辑和删除操作。学生则可以在竞赛开放期间完成报名和参赛。

在排行榜实现上，系统根据用户提交记录对通过题数、总提交次数和罚时进行汇总，并生成实时榜单。若后续需要扩展冻结榜、滚榜展示或更复杂的竞赛规则，现有结构也保留了一定扩展空间。该模块的实现增强了系统的实践教学属性，使平台不仅服务于日常练习，也能够支持更具挑战性的课程活动。

竞赛功能主要由 `ContestController` 和相关服务模块支撑，系统支持竞赛时间区间、题目关联、报名记录、排行榜统计以及罚时策略扩展，满足课程练习赛与基础校内赛场景。

5.4 讨论区模块实现

实现发帖、详情页、评论与楼中楼回复。管理员可执行审核，保障社区内容质量。

讨论区模块用于支撑学生之间以及学生与教师之间的学习交流。用户可以围绕题目难点、算法思路、调试问题和比赛经验发布帖子，并在帖子详情页中进行评论和回复。通过楼中楼回复设计，系统能够更清晰地表达讨论上下文，提升交流效率。

在实现上，系统通过帖子与评论两类核心实体组织讨论内容，并结合用户身份信息展示发帖人与评论人。为了保证社区内容秩序，管理员还可对帖子进行审核和管理。讨论区模块的加入，使系统从单纯的评测平台进一步拓展为具有学习交流属性的教学辅助平台。

社区交流部分由 `DiscussionController`、`DiscussionCommentController` 与 `SocialController` 共同支撑，形成发帖、评论、回复和互动的内容闭环，有助于学生在解题之后开展经验总结与同伴互助。

5.5 后台管理与统计实现

实现系统配置、操作日志、趋势统计等功能，用于日常运维与教学管理支持。

后台管理模块主要服务于系统管理员和部分教师用户。系统通过配置管理接口维护平台运行参数，并通过操作日志记录关键管理行为，以便在出现异常时进行追踪和排查。同时，平台还提供一定的统计能力，用于展示用户增长、提交量变化、题库数量变化等数据，为教学管理和系统运维提供辅助信息。

从系统整体运行角度看，后台管理模块虽然不是学生最常使用的部分，但对平台稳定

运行具有重要意义。通过日志和监控信息，管理者可以及时发现问题、分析运行状态，并根据实际情况优化系统配置，从而提升平台长期可用性。

系统后台提供 AdminSystemController、TeacherAnalyticsController、SystemController 等接口，用于查看系统监控、教学统计、配置项和运行状态，为系统维护与教学管理提供数据依据。

第 6 章 系统测试

6.1 测试环境与测试方法

测试环境为 Windows + JDK17 + MySQL8 + Docker。采用功能测试、接口测试与回归测试相结合方式。

为验证系统的可用性与稳定性，本文在本地部署环境下完成测试工作。测试环境包括 Windows 操作系统、JDK17、MySQL8、Redis 以及 Docker 判题环境，前后端服务分别启动后，通过浏览器访问和接口脚本对系统进行综合验证。

测试方法上，主要采用功能测试、接口测试和回归测试相结合的方式。功能测试用于验证各页面和业务流程是否符合预期；接口测试用于验证后端核心接口的输入输出和鉴权逻辑；回归测试则用于在模块修改后复查已有功能是否受到影响。通过多种测试方式结合，可以较全面地评估系统当前实现情况。

项目仓库中已提供后端检查脚本、接口冒烟脚本和性能冒烟脚本，说明系统测试不仅覆盖页面功能，也覆盖了接口可用性与基础性能验证，具有一定工程实践基础。

6.2 功能测试与结果分析

重点覆盖登录、题库、提交、竞、讨论和后台管理。结果显示主流程均能稳定执行，功能符合预期。

在用户登录与权限测试中，系统能够正确区分游客、学生、教师和管理员的访问范围，未授权请求会被拒绝或跳转登录。题库模块测试表明，题目列表、题目详情、测试点维护和提交入口均能正常工作。提交评测测试中，系统能够根据不同代码结果返回对应状态，并支持查看历史提交记录。

竞赛模块测试主要覆盖竞赛创建、报名和排行榜展示等场景，结果表明系统可以在竞赛期间正常统计成绩并展示榜单。讨论区测试中，发帖、查看详情、评论和回复功能均可正常执行。后台管理测试则验证了系统配置查看、日志记录和统计展示等能力。总体来看，各主要功能链路均可稳定运行，达到了毕业设计阶段的预期要求。

6.3 性能测试与安全测试

在教学规模并发下，系统可保持可用。判题高峰存在队列堆积，后续通过并行 worker 与缓存优化提升吞吐。安全方面完成鉴权、参数校验与运行隔离。

在性能层面，系统针对常见教学规模下的访问进行了基础验证。结果表明，在题目浏

览、普通接口访问和中低强度提交场景中，平台能够维持正常响应。但当提交请求在短时间内集中增加时，判题链路会出现一定程度的排队现象，这说明评测任务调度与并发执行能力仍有进一步优化空间。

在安全层面，系统已实现登录鉴权、角色控制、参数校验和判题隔离等基础措施。业务接口不会向未授权用户开放敏感操作，判题过程也通过隔离环境执行用户代码，降低了恶意代码直接影响业务服务的风险。虽然当前安全机制已满足毕业设计阶段的基本需求，但在生产级场景下仍可继续增强，例如引入更细粒度的资源限制、敏感操作告警与异常行为检测等机制。

6.4 测试结论

系统已满足毕业设计阶段目标，具备课程训练与基础竞赛应用能力。

综合各项测试结果可以看出，系统已经基本完成了从题库训练、在线评测到竞赛管理和后台维护的主要功能实现，核心业务链路运行正常，能够满足程序设计课程教学和基础竞赛组织的应用需求。测试过程中的问题主要集中在高峰评测吞吐和个别交互细节方面，不影响系统作为毕业设计成果进行展示与说明。

第 7 章 结论

系统完成了“题库训练—自动评测—竞赛组织—讨论交流—后台治理”全链路实现，能够为程序设计课程提供可落地的教学支撑平台。后续将继续完善测试数据规模、优化并发判题性能、增强安全策略，并推进部署工程化。

本文围绕高校程序设计课程教学需求，设计并实现了一套基于 **Spring Boot** 的程序设计评测系统。系统以学生、教师和管理员三类角色为核心，较完整地实现了题库管理、在线提交、自动判题、竞赛组织、讨论交流、系统配置与日志管理等功能，并形成了从训练到反馈再到统计分析的基础教学闭环。通过前后端分离架构、分层业务设计和关系型数据库建模，系统具备了较好的结构清晰性和一定的后续扩展能力。

从测试结果来看，系统在课程训练和基础竞赛场景下能够稳定运行，验证了整体设计方案的可行性。与此同时，本文也认识到系统仍存在一定改进空间，如判题并发能力仍有待提升，题库内容与测试数据规模还可进一步丰富，讨论区管理与学习数据分析功能也有进一步深化的可能。后续工作将围绕系统性能优化、功能细化和工程化部署持续推进，使平台在教学实践中发挥更大价值。

参考文献

- [1] 钟耀章, 桂琼. ACM 竞赛在线测评系统设计与实现[J]. 无线互联科技, 2020(18).
- [2] 曾棕根. 源程序在线评测系统技术改进[J]. 计算机工程与应用, 2011, 47(4):68-71.
- [3] 基于 Vue 和 SpringBoot 的 C 语言程序在线测评系统的设计与实现[J]. 唐山师范学院学报, 2023(03).
- [4] 基于 Spring Boot 的课程评价系统设计与实现[J]. 计算机应用研究, 2024(01).
- [5] 范佳杰, 王金庆, 刘钦. 编程考试系统代码质量度量及相似度检测子系统的设计与实现[D]. 南京大学, 2020.
- [6] 尤枫, 史晟辉, 赵瑞莲. 编译程序在线评测系统的实现[J]. 实验室研究与探索, 2010, 29(12):69-72.
- [7] 中华人民共和国国家标准. 信息与文献参考文献著录规则: GB/T 7714—2015[S].

附录

附录 1:关键接口清单

1. 认证接口:登录、注册、获取用户信息。
2. 题库接口:题目列表、题目详情、测试点管理。
3. 提交接口:代码提交、提交列表、提交详情。
4. 竞赛接口:竞赛列表、报名、排行榜。
5. 讨论接口:帖子列表、发帖、评论、回复。

控制器层当前包括 AuthController、ProblemController、SubmissionController、ContestController、DiscussionController、DiscussionCommentController、SocialController、TeacherAnalyticsController、UserController 等接口入口。

附录 2:数据库表结构补充

列出核心表及关键索引,正式稿中建议附建表 SQL 片段。

数据库迁移脚本已按版本方式管理,目前包含竞赛冻结榜与罚时配置、讨论回复与社交能力、用户头像字段、提交样例结果、帖子审核等增量变更记录,便于后续维护与演进。

致 谢

在课题实施和文稿撰写过程中，感谢指导教师在选题、设计、测试和改稿阶段提供的指导与建议；感谢同学在系统联调和测试阶段提供的帮助。

附 录

1. 初始化数据库。
2. 配置后端 `application.yml`。
3. 启动后端服务。
4. 构建并部署前端。
5. 启动 Docker 判题环境。